

第 4 章 函 数

函数是能完成某一特定功能并且可以反复使用的程序模块，也称为子程序。一个 C++ 程序由一个主函数和若干个子函数组成。除主函数外，其他函数是不能独立运行的。函数有各种形态，这取决于函数的定义和调用。本章将介绍函数的定义及调用、函数原型、函数重载、函数参数的缺省、函数模板等的作用以及函数与指针的关系、函数和变量的作用域。在掌握函数定义形式和调用方法的基础上，重点要领会函数间数据的传递机制。

4.1 函数概述

一个较为复杂的系统往往需要划分为若干个子系统，然后对这些子系统进行开发。这些子系统在 C++ 程序中就是函数。这是程序设计中一个很好的策略。通过把一个较为复杂的程序划分为若干个相互关联、功能独立、任务简单的模块，使得复杂的问题逐步简化。这样的好处是减小编程难度，利于实现参数化，并节省程序开发时间、便于管理和阅读，而且各模块中的错误不易扩散到其他模块，从而提高了程序的正确性和可靠性。

在面向过程的结构化程序设计中，函数是模块划分后的基本单位，是对问题处理过程的一种抽象。在面向对象的程序设计中，函数具有更重要的作用，并且从形式到功能都有着革命性的变化。

在 C++ 中，一个程序无论由多少个源程序文件组成，其中有且仅有一个主函数 `main()`。`main()` 函数在源程序中的位置任意，可以把它放在其他函数之前，也可以放在其他函数之后。不管 `main()` 函数存放在哪个源程序文件中，也不管在该源程序文件中的什么位置上，整个程序总是从 `main()` 函数开始执行。当 `main()` 函数执行结束时，整个程序就正常结束了。当然，也可能出现由于各种原因在其他函数中异常终止的现象。

函数分为两种，即系统库函数和用户定义函数。

系统库函数是一种由系统提供的、在程序中可直接使用的函数。用户要使用库函数，只需要在程序中包含库函数的头文件即可。详细内容见 4.10 节。

用户定义函数是程序设计者在程序中定义的函数。程序中使用的函数要先定义。使用函数叫做调用函数，或称函数的调用。一个函数可以调用其他函数，也可以被其他函数调用，还可以直接或间接地调用自身(递归调用)。

4.2 函数的定义和调用

4.2.1 函数的定义

定义函数包括定义函数的名称、函数的返回类型、函数参数的个数和类型以及用语句实现的函数的功能。

函数定义的一般形式为：

```
类型名 函数名(形式参数表)
{
    语句序列
}
```

类型名是指函数被调用后返回值的类型。它既可以是基本数据类型和构造数据类型，也可以是指针或引用。函数的返回值由函数中的 `return` 语句实现。如果一个函数没有返回值，返回类型指定为 `void`。

函数名由用户命名，与变量名的命名方法一样，是一种标识符，一般函数名最好做到“见名知意”，例如用 `maxint` 做函数名，表示该函数是求整数的最大值，用 `averagedouble` 做函数名，表示求 `double` 类型数据的平均值等。

函数名后的形式参数表必须用圆括号括起来。每个形式参数必须指明类型。形式参数之间用逗号隔开。形式参数用来接收调用时传递给该函数的实际参数。如果调用时没有数据传递，可以在形式参数表位置写上 `void`，也可以是空表，但圆括号不能省略。

用一对花括号 `{}` 括起来的语句序列称为函数体。函数体内给出了完成函数功能所要执行的各种语句。不允许在一个函数体内定义另一个函数。

当一个函数执行后需要将一个值带回到调用函数时，可以使用 `return` 语句。`return` 语句的形式为：

`return 表达式;`

或

`return;`

`return` 语句中的表达式值就是所求函数值。此表达式值的类型要与定义函数时函数名前的类型一致。若类型不一致，则以函数类型为准，由系统自动进行转换。当程序执行到 `return` 语句时，退出被调用函数，返回到调用该函数的地方并带回函数值。一个函数可以有多个 `return` 语句，分别用于不同条件下的返回。若 `return` 是函数中的最后一条语句并且该函数没有返回值时，`return` 可以省略，函数体的右花括号有返回的功能。

例 4.1 编写求 x^n 的函数。

```
long power(int x, int n)
{ long p=1;
  for(int i=1; i<=n; i++)
    p=p*x;
  return p;
}
```

该函数有两个参数 `x` 和 `n`。函数体内用一个循环语句将 `x` 连续乘 `n` 遍，实现求 x^n ，结果存放在 `p` 中。执行到 `return p` 语句时，结束函数的执行，返回到调用该函数的调用处，并将结果带回到调用处。

例 4.2 无形参和无返回值的函数。

```
void alphabet(void)
{ cout<<"ABCDEFGHIJKLMNOPQRSTUVWXYZ\n";
  return; //可省略
}
```

执行该程序显示 26 个英文大写字母，显示后返回到调用函数。该函数中的 `return` 语句可以省略。

例 4.3 有多个 `return` 语句的函数。

```
int max(int a1, int a2) //返回 a1 和 a2 中的最大值
{ if(a1>a2) return a1;
  else return a2;
}
```

4.2.2 函数的调用

使用一个函数就是调用该函数。函数定义后可以被其他函数调用，既可以在主函数 `main()` 中调用，也可以在另一个定义函数中调用。一个函数定义后可以被其他函数多次调用。

函数调用形式为:

函数名(实参表)

调用函数时实参表中应提供与被调用函数的形参表中形参个数和类型相同的实际参数。若实际参数为多个,中间用逗号分隔。实际参数可以是常量、变量、函数调用、表达式等。实际参数是对被调用函数形参的初始化。初始化是按位置进行的,因此要求实参和形参在个数和类型上要对应,既第一个实参对应第一个形参,第二个实参对应第二个形参,等等。

按照被调用的函数是否有返回值,函数调用有两种不同的形式。第一种形式是函数调用作为一个量出现在表达式中,此时的被调用函数必须有返回值。第二种形式是调用函数单独作为一个语句使用,此时调用的是没有返回值的函数。例 4.3 是第一种形式的调用。

例 4.4 求 x^3+x^5 的值。

```
#include <iostream.h>
int power(int x, int n);           //函数原型
void main()
{  int x, s;
  cout<<"输入 x 值:";  cin>>x;
  s=power(x,3)+power(x,5);
  cout<<"s="<<s<<endl;
}
int power(int x,int n)
{  int i,p=1;
  for(i=1; i<=n; i++) p=p*x;
  return p;
}
```

执行结果为:

```
输入 x 值:2
s=40
```

以上程序中的 `power()` 函数完成求 x^n 的功能。在 `main()` 函数中,通过语句 `s=power(x,3)+power(x,5)` 两次调用 `power()` 函数。第一次调用时将实参 `x` 传给 `power()` 函数中的形参 `x`,将 3 传给形参 `n`,然后执行 `power()` 函数体,求出 x^3 放在变量 `p` 中,执行到 `return p` 时,返回到主函数,将结果值代替 `power(x,3)`。第二次调用时,将实参 `x` 和 5 传给形参 `x` 和 `n`,执行后得到 x^5 ,执行到 `return p` 时将 x^5 的结果 `p` 返回到主函数 `power(x,5)` 处。最后将两次的结果值相加后存放在 `s` 中。

函数的调用过程大致可以分为以下几步:①实参与形参结合,将实参的值传递给形参,如果是无参函数,此步不执行;②执行被调用函数体中的各语句;③当执行到 `return` 语句或函数体后面的右花括号时,返回到调用函数,并将结果带回,若无结果带回,则直接返回到调用函数,继续执行调用函数下面的语句。

下面是调用单独作为一个语句出现的例子。

例 4.5 无返回值函数调用举例。

```
#include <iostream.h>
void psum(int a1, int a2)
{  cout<<a1*a1+a2*a2<<endl;  }           //输出两个数平方的和
void main()
{  int a,b;
  cout<<"输入 a、b:";  cin>>a>>b;
```

```

        psum(a,b);                //求 a2+b2
        psum(a+b, a-b);           //求(a+b)²+(a-b)²
    }

```

执行结果为:

```

输入 a、 b:3  5
34
68

```

调用也可以作为另一个函数调用的参数,例如调用例 4.1 求 x^n 的函数。主函数如下:

```

int main()
{   int x,i;
    cout<<"输入 x 值:"; cin>>x;
    for(i=1; i<=10; i++)
        cout<<power(x, i)<<endl;
    long m=power(power(x, 2), 3);
    cout<<m<<endl;
    return 0;
}

```

for 循环中的 `cout<<power(x, i)<<endl` 完成调用并输出函数返回值。当 for 循环的 i 由 1 到 10 变化时,分别调用 10 次 `power()` 函数,求出 x^1 到 x^{10} 的值并输出。语句 `long m=power(power(x, 2), 3)` 对函数 `power()` 进行两次调用,第一次调用的返回值作为第二次调用的参数,计算的是 $(x^2)^3$ 的值。

4.3 函数原型

系统在处理函数调用时,需要知道函数的类型,函数名,函数参数的个数、类型及参数的顺序,以便确认函数调用时语法和语义是否正确。以上信息组成函数的原型。C++ 声明函数就是函数原型。函数原型的一般形式为:

类型名 函数名(形参表);

函数原型和定义函数时函数的第一行基本一致。函数原型的参数表中可以不包含参数的名字,只包含参数的类型及顺序。如例 4.4 中的第二行,即为函数原型。该原型也可以写成 `int power(int, int);`。

函数原型是一条程序语句,最后用“;”号结束。

从格式上看,函数定义第一行的形式与函数原型是一致的。因此如果函数定义在函数调用之前,函数定义本身就是函数原型,如例 4.5 中函数 `psum()` 的定义在 `main()` 函数的前边,不需再给出函数原型。这样做虽然可以用函数定义起到函数原型声明的作用,但当组织由多个函数组成的程序时,就要在安排函数的顺序上费一些时间。例如程序有如下三个函数:

```

void f1() {...}
void f2() { ...f1();           //f2 函数中调用 f1 函数可以
                ...f3()...}    //出错, 因为 f3 函数在前边没有说明
void f3() {...}
void main() {...}

```

以上程序 `main()` 函数调用 `f1()`、`f2()` 和 `f3()` 函数都可以,但 `f1()`、`f2()` 和 `f3()` 互相调用时就要注意顺序。当在组织一个程序时,好的编程习惯是:无论函数的定义在什么地方,在程序的前部都给出函数原型声明。将以上程序进行重新组织,得

```

void f1();           //函数原型
void f2();           //函数原型
void f3();           //函数原型
void main() {...}
void f1() {...}
void f2() {...}
void f3() {...}

```

这样，无论上述程序中函数是怎样的调用关系，都不会出现问题。

例 4.6 分析该程序并输出结果。

```

#include <iostream.h>
int f1(void),f2(void),f3(void);
void main()
{   cout<<"this is main()\n";
    cout<<f1()<<endl;
}
int f1(void)
{   cout<<"this is f1()\n";
    cout<<f3()<<endl;
    return 1;
}
int f2(void)
{   cout<<"this is f2()\n";
    return 2;
}
int f3(void)
{   cout<<"this is f3()\n";
    cout<<f2()<<endl;
    return 3;
}

```

执行结果为:

```

this is main()
this is f1()
this is f3()
this is f2()
2
3
1

```

4.4 参数的传递机制

函数在没有被调用时，函数的形参并不占有实际的内存空间，当然也就没有具体的值。只有在调用函数时，被调用函数的形参才分配存储空间。在对函数进行调用时，必须提供与形参类型和个数一致的实参。在 C++ 中将实参的值传递给形参时有两种传递机制。一种是值的传递(passing by value)，称为值调用。在值的传递中，形参又可有变量作形参和地址作形

参两种形式。另一种是引用传递(passing by reference)，称为引用调用。

4.4.1 函数的值调用

这种调用的特点是将实参的值传递给形参。值传递时形参是变量。由于变量分为存放值的一般变量和存放地址的地址变量(指针)，因此函数的值调用又有传值调用和传地址调用。

1.传值调用

在传值调用机制中，当函数被调用时，形参分配存储空间，并将实参的值按位置拷贝(传递)到相应的形参中。在执行被调用函数时，访问拷贝后的形参，与实参没有关系。这种传递机制的特点是，形参的变化不会影响到实参。

例 4.7 传值调用举例。

```
#include <iostream.h>
void swap(int x, int y);           //函数原型
void main()
{  int a(15), b(18);
  cout<<"主函数第 1 次输出: a="<<a<<"   "<<"b="<<b<<endl;
  swap (a, b);
  cout<<"主函数第 2 次输出: a="<<a<<"   "<<"b="<<b<<endl;
}
void swap(int x, int y)
{  int temp;
  temp=x; x=y; y=temp;
  cout<<"swap 函数输出: x="<<x<<"   "<<"y="<<y<<endl;
}
```

执行结果为:

```
主函数第 1 次输出:a=15    b=18
swap 函数输出:x=18    y=15
主函数第 2 次输出:a=15    b=18
```

从该程序的结果看，main()函数调用 swap()函数时，将实参 a 的值 15 传递给形参 x，b 的值 18 传递给 y。函数 swap 实现 x 和 y 两个数的交换。交换后 x 是 18，y 是 15。调用后返回到主函数，主函数输出的 a 仍为 15，b 仍为 18。可见形参 x 和 y 的变化对实参没有影响。

2.传地址调用

在传地址调用中，被调用函数的形参是指针，调用函数的实参给出的是地址值。调用时，将实参的地址值传递给对应的形参指针，使形参的指针也指向实参的变量。这种调用方式与传值调用不同的是实参传递给形参的是调用函数中变量的地址，而不是变量中的值。在被调用函数中，改变形参指针所指向变量的内容时，间接地改变了实参地址中的内容。这种传递机制的特点是，通过改变形参所指向变量的值来影响实参。

例 4.8 传地址调用举例。

```
#include <iostream.h>
void swap(int *px, int *py);       //函数原型
void main()
{  int a(15), b(18);
  cout<<"主函数第 1 次输出: a="<<a<<"   "<<"b="<<b<<endl;
  swap(&a, &b);                    //实参是变量的地址
```

```

    cout<<"主函数第 2 次输出: a="<<a<<"    "<<"b="<<b<<endl;
}
void swap(int *px, int *py)
{   int temp;
    temp=*px;  *px=*py;  *py=temp;
    cout<<"swap 函数输出: *px="<<*px<<"    "<<"*py="<<*py<<endl;
}

```

执行结果为:

```

主函数第 1 次输出: a=15    b=18
swap 函数输出: *px=18    *py=15
主函数第 2 次输出: a=18    b=15

```

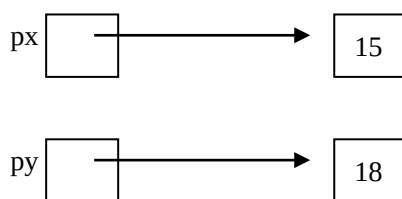


图 4.1 传地址调用示意图

在 main()函数中调用 swap()函数时，参数是变量 a 和 b 的地址。调用执行时，将 a 的地址传递给指针 px，将 b 的地址传递给指针 py。参数传递后的结果见图 4.1。在函数 swap()中交换 px 指针和 py 中的内容，实际交换的是 a 和 b 的内容。最后在 main()中再输出 a 和 b 的内容时，就是交换后的内容。

4.4.2 函数的引用调用

前面已介绍过引用的概念。引用是给已知变量起个别名。引用作形参时，可以被认为是实际参数的一个别名，传递后形参和实参对应的是同一个量。当函数被调用时，形参的值发生了变化，那么与形参对应的实参的值也就发生了变化。也就是说，对形参的任何操作都直接作用于实参。

使用引用作函数参数时，实参要用变量名，而对应的形参用引用符“&”。仍以上述两个数的交换为例，用引用实现。

例 4.9 引用调用举例。

```

#include <iostream.h>
void swap(int &x, int &y);          //函数原型说明
void main()
{   int a=15,b=18;
    cout<<"主函数第 1 次输出: a="<<a<<"    "<<"b="<<b<<endl;
    swap(a, b);
    cout<<"主函数第 2 次输出: a="<<a<<"    "<<"b="<<b<<endl;
}
void swap(int &x,int &y)
{   int temp;
    temp=x;  x=y;  y=temp;
}

```

执行结果为:

主函数第 1 次输出: a=15 b=18

主函数第 2 次输出: a=18 b=15

从运行结果可以看出, 通过调用 `swap()` 函数实现了两个数的交换。函数 `swap()` 的两个形参被说明为引用。调用时, `a` 和 `x` 都是存放数值 15 的变量名, `b` 和 `y` 都是存放 18 的变量名。当 `x` 和 `y` 的内容交换后, 交换的自然也是 `a` 和 `b` 的内容。

使用引用做函数形参起到了传地址调用的作用, 但它比传地址调用更方便、直观, 因此在 C++ 中常常用引用作函数的形参, 达到通过形参改变实参的目的。

一个函数只能返回一个值。如果需要从函数返回多个值时, 可以用引用实现, 见以下函数:

```
void complex(double real, double imag, double &realResult, double &imagResult)
{   realResult=real*real;
    imagResult =imag*imag;
}

void function()
{   double r=3.5, v=-8.0, freal, fimag;
    complex(r, v, freal, fimag);
    cout<<freal<<" "<<fimag<<endl;
}
```

函数 `function()` 在调用 `complex()` 函数时, 将变量 `freal` 和 `fimag` 传递给引用型形参 `realResult` 和 `imagResult`, 使得 `function()` 函数中的 `freal` 与 `complex()` 函数中的 `realResult`、`function()` 函数中 `fimag` 与 `complex()` 函数中 `imagResult` 共享存储空间。当在 `function()` 函数中调用完 `complex()` 函数后, 输出 `freal` 和 `fimag` 时, 输出的是 `realResult` 和 `imagResult` 的值。

4.4.3 数组作参数

当数组作为函数的参数时, 也是采用传地址的方法, 这样就避免了成批数据在形参和实参之间进行复制。数组作为函数参数时, 有以下几种形式。

1. 形参和实参都是数组

被调用函数的形参写数组的定义, 调用函数的实参直接写数组名。此时形参和实参共用内存中的同一个数组, 形参和实参处理的是同一个数组。当被调用函数中改变数组中某些元素的值时, 改变的就是调用函数中该数组元素的值。

例 4.10 将数组 `s` 中的数按由小到大顺序重新排列。

```
#include <iostream.h>
void sort (int a[10]);           //函数原型
void main()
{   int s[10]={2,3,5,1,7,9,0,4,8,6};
    sort(s);                     //调 sort 函数, 实参是数组名
    for(int i=0; i<10; i++) cout<<s[i]<<" ";    //输出排序后的结果
    cout<<endl;
}

void sort(int a[10])
{   int temp;
    for(int i=0; i<9; i++)
        for(int j=i+1; j<10; j++)
```

```
        if (a[i]>a[j]) { temp=a[i];  a[i]=a[j]; a[j]=temp; }
    }
}
```

执行结果为:

```
0  1  2  3  4  5  6  7  8  9
```

程序说明如下:

①用 sort()函数实现排序。排序方法是先从 10 个数中选出一个最小的放在 a[0]中,再从剩下的 9 个中选出一个小的放在 a[1]中。如此下去,直到排序完毕。

②sort()函数中的数组 a 中的值即为 main()函数调用时 s 数组中的值。a 和 s 是同一个数组的存储空间。排序完成后的结果仍在数组原来的存储空间。因此 main()中输出的即为排序后的结果。

③sort()函数的形参也可以写成(int a[]), 实际大小由实参数组确定。

2.形参是指针, 实参是数组名

由于数组名表示数组的首地址, 将实参的数组名给到形参的一个指针, 在被调用函数中通过指针处理该数组。

例 4.11 形参是指针, 实参是数组名举例。

```
#include <iostream.h>
void fun (int *p);           //函数原型
void main()
{ int  a[] = {2,4,6,8,10 };
  fun(a);                    //调 fun()函数
  for(int i=0; i<5; i++) cout<<a[i]<<" "; //输出数组中的元素
  cout<<endl;
}
void fun(int *p)
{ int t;
  for(int *q=p+4 ; q>=p; q--,p++)
  { t=*q; *q=*p; *p=t; }
}
```

执行结果为:

```
10  8  6  4  2
```

该程序中 fun()函数的形参 p 是个指向整型的指针, 调用时, 让它指向数组 a 的首地址。实际上这是个传地址调用。在 fun()函数中通过指针的移动将数组 a 中元素的内容进行交换。main()输出交换后的结果。

3.二维数组作参数

二维数组名作实参时, 被调用函数的形参可以有以下形式:

```
类型 (*px)[列数]      (px 是行指针)
类型 x[][列数]        (x 是省略行数的二维数组)
类型 x[行数][列数]    (二维数组)
```

二维数组作参数时, 无论采用以上哪种形式, 调用函数和被调用函数处理的都是同一数组。

例 4.12 求二维数组中每行元素的最大值及最大值所在列的位置。

程序中用 fmax()函数求每行最大值及最大值所在位置。下面分别给出 fmax()函数几种不同的参数形式。

形式一:

```

#include <iostream.h>
const int N= 5;
void fmax(int (*p)[N]);          //函数原型，行指针作形参
void main()
{   int a[N][N]={0},{0,-5,3,6,12},{1,-1,9,7,0},{0,1,3,-5,8},{0,1,-1,-2,0}};
    fmax(a);                      //调 fmax()函数，数组名作实参
}
void fmax (int (*p)[N])
{   int i, j, r, c, hmax;
    for(i=0; i<N; i++)
    { hmax=*(*(p+i)+0);
      r=i; c=0;
      for (j=0; j<N; j++)
          if (*(*(p+i)+j)>hmax) { hmax=*(*(p+i)+j); c=j; }
      cout<<r<<" " <<c<<" " <<"max="<<hmax<<endl;
    }
}

```

执行结果为:

```

0 0 max=0
1 4 max=12
2 2 max=9
3 4 max=8
4 1 max=1

```

形式二:

```

#include <iostream.h>
const int N= 5;
void fmax(int *p);              //函数原型，普通指针作形参
void main()
{   int a[N][N]={0},{0,-5,3,6,12},{1,-1,9,7,0},{0,1,3,-5,8},{0,1,-1,-2,0}};
    fmax(a[0]);                  //调 fmax()函数，初始地址作实参
}
void fmax(int *p)
{   int i, j, r, c, hmax;
    for(i=0; i<N; i++)
    { hmax=*(p+0);
      r=i; c=0;
      for(j=0; j<N; j++)
          if (*(p+j)>hmax) { hmax=*(p+j); c=j; }
      cout<<r<<" " <<c<<" " <<"max="<<hmax<<endl;
      p+=N;                      //修改指针，指向下一行
    }
}

```

形式三:

```

#include <iostream.h>

```

```

const int N= 5;
void fmax(int p[][N]);    //函数原型，二维数组作形参 (可省略一维大小)
void main()
{   int a[N][N]={0},{0,-5,3,6,12},{1,-1,9,7,0},{0,1,3,-5,8},{0,1,-1,-2,0}};
    fmax(a);              //调 fmax()函数，数组名作实参
}
void fmax(int p[][N])
{   int i, j, r, c, hmax;
    for(i=0; i<N; i++)
    { hmax=p[i][0];
      r=i; c=0;
      for(j=0; j<N; j++)
          if(p[i][j]>hmax){ hmax=p[i][j]; c=j; }
      cout<<r<<" "<<c<<" "<<"max="<<hmax<<endl;
    }
}

```

在使用 main()函数时，前面例题没有使用参数。实际上在系统调用 main()函数开始执行程序时，可以由操作系统向它传递参数，此时 main()函数参数的形式为：

```
main(int argc, char *argv[ ])
```

其中 argc 是整型变量，它存放命令行中包括程序名(命令名)在内的所有参数的个数。argv 是指针数组，长度为 argc。它的元素分别指向命令行上各参数。

4.5 嵌套调用和递归调用

1. 嵌套调用

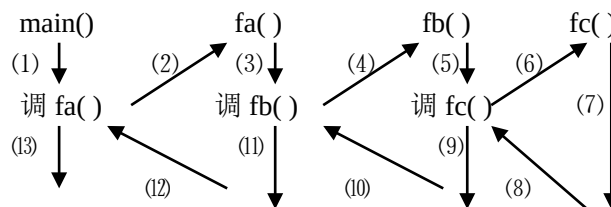


图 4.2 嵌套调用示意图

嵌套调用指的是在调用 fa()函数的过程中又调用 fb()函数，在调用 fb()函数时又调用 fc()函数.....函数的嵌套调用过程见图 4.2。图中箭头方向和(1)、(2)、.....就是程序的执行方向。

例 4.13 编写计算下列函数的程序：

$$f(x, y) = \frac{s(x)}{s(y)}$$

其中
$$s(n) = \sum_{i=1}^n p(i)$$

$$p(i) = i!$$

程序如下:

```
#include <iostream.h>
double f(int x1, int y1);
long s(int n);
long p(int i);
void main()
{   int x,y;
    cout<<"输入 x 、 y:";   cin>>x>>y;
    cout<<"f (x,y)="<<f(x,y)<<endl;
}
long p(int i)
{   for(int n=1, long k=1; n<=i; n++)   k=k*n;
    return k;
}
long s(int n)
{   for(int j=1, long sum=0; j<=n; j++) sum=sum+p(j);
    return sum;
}
double f(int x1, int y1){   return   (double)s(x1)/(double)s(y1);   }
```

执行结果为:

输入 x 、 y:4 5

f(x,y)=0.215686

以上程序 main()函数调用 f()函数求 $s(x)/s(y)$, f()函数调用 s()函数求 $1!+2!+3!+\dots+j!$, s()函数又调用 p()函数求 $i!$ 。

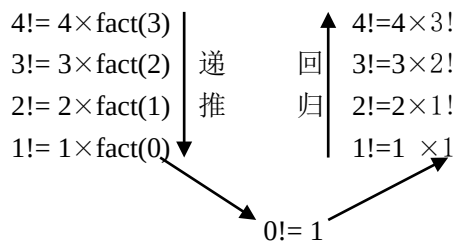
2.递归调用

递归调用又称自调用,即在定义函数的内部直接或间接地调用自己。使用递归算法编写的程序结构简练、可读性强,但执行时时间和空间上的开销都比较大。

在递归函数中,由于程序反复地自己调用自己,每一次调用时参数值都发生变化。为防止自调用无休止地进行下去,在函数内必须设置调用终止的条件,当条件满足时能从函数中返回。

例 4.14 用递归函数实现求 $n!$ 。

```
#include <iostream.h>
long fact(int n);
void main()
{   int n;
    cout<<"输入正整数 n:"; cin>>n;
    cout<<"n!="<<fact(n)<<endl;
}
long fact(int n)                                //递归函数
{   long h;
    if(n>0) h=n*fact(n-1);
    else h=1;
    return h;
}
```



当输入 $n=4$ 时，执行结果为 $n!=24$ 。程序的执行过程见图 4.3。

递归调用过程可以分为两个阶段:第一阶段称为递推，将原有问题逐步分解为一个和原有问题处理方法相同的更简单问题，直到分解出的问题已知。第二阶段称为回归，这个阶段从已知的条件出发，按照递推的“逆”过程逐步求值，最终到达递推的开始处，得到结果。

在递归调用过程中系统不断建立临时单元存放各次需保存的值，直到自调用结束;在返回时，不断释放内存单元。编译系统处理递归调用时，临时单元用栈来实现，调用时将各次需保存的值依次压入栈区。返回时，依次退栈。因此，递归调用所需的存储空间和运行时间要比一般函数调用多。此外，误用递归调用可能造成非常隐蔽的错误。

4.6 函数与指针

4.6.1 函数返回指针和返回引用

函数的类型是由函数被调用后返回值的类型标识的。函数返回值可以是 `int` 类型、`char` 类型或 `double` 类型等基本类型，分别称为整型函数、字符型函数和双精度型函数等。函数的返回值也可以是一个指针或引用。

1. 指针函数

当一个函数被调用时，若函数返回值是一个指针(地址)，则称该函数为指针函数。指针函数主要用在函数结束时需要大量的数据从被调用函数返回调用函数的情况。当一个函数被调用后，可以返回一个值。若有多个顺序存放的值(例如数组、字符串)需要返回时，可以用指针函数处理。

指针函数的定义形式为:

类型 *函数名(参数)

例 4.15 输入 0—6 数字，输出对应的英文星期表示

```
#include <iostream.h>

char *fun( int  n)           //定义指针函数
{ static char *pstr[]={"sun","mon","tue","wed","thu","fri","sat"};
  return pstr[n];
}

int main()
{  int n;
   cout<<"输入 0—6:"; cin>>n;
   if(n<0||n>6) cout<<"输入错!\n";
   else cout<<fun(n)<<endl;
   return 0;
}
```

执行结果示例 1:

输入 0—6: 3

wed

执行结果示例 2:

输入 0—6: 8

输入错!

程序说明如下:

①程序中 `char *fun( int n)` 为函数定义。函数名 `fun( )` 前有“*”，表示函数调用后返回的是个指针。

②在 `fun( )` 函数的字符型指针数组 `pstr` 中存放表示星期的 7 个字符串。`pstr` 数组中的每个元素是指向字符串的指针，因此 `pstr[n]` 返回的是个指针。

③在 `main( )` 函数中将调用 `fun( )` 函数的结果（字符串）输出。

使用返回指针的函数时要注意，不能把局部作用域内数据的地址作为返回值，因为函数返回后，函数的局部变量占有的空间被释放，引用已消失空间的地址是危险的。因此在 `fun( )` 函数中将字符型指针数组 `pstr` 声明为 `static`(静态)，这样在函数返回后静态变量不会消失，对地址的引用是安全的。

2. 返回引用

函数也可以返回引用。系统处理返回引用与处理返回值是有区别的。处理函数返回值时，系统生成一个返回值的副本(或称临时变量)，由这个临时变量给到调用函数的接受变量。而处理返回引用时，不生成副本，直接将引用的值给到调用函数。

由于引用只是另一个量的别名，因此返回引用要返回在作用域范围内的变量(或对象)的引用。因为一个变量(或对象)一旦退出所在的作用域，对它的任何引用也就毫无意义。

例 4.16 函数返回引用。

```
#include <iostream.h>
int &f1(int &a1);           //函数 f1( )返回引用
int f2(int a2);
void main()
{   int x=2;
    int b1=f1(x);
    int b2=f2(x);
    cout<<"b1="<<b1<<endl;
    cout<<"b2="<<b2<<endl;
    cout<<"x="<<x<<endl;
}
int &f1(int &a1)
{   a1+=a1;   return a1; }
int f2(int a2)
{   a2+=a2;   return a2; }
```

运行结果为:

b1=4

b2=8

x=4

若将 `f1( )` 函数作以下修改:

```
int &f1(int &a1)
{   int aa=a1+a1;
```

```

        return aa;                //出现警告性错误
    }

```

因为 aa 是局部变量，调用 f1() 函数结束后，aa 变量占用的空间消失，对 aa 的引用也变得没有意义。这种情况在 VC 中出现警告性错误，在 BC 中是编译性错误。因此编程时要注意避免。

4.6.2 函数指针

数组名是表示数组首地址的地址常量。与此类似，在 C++ 中函数名代表函数的入口地址的地址常量。因此可以声明一个指向函数的指针，让这个指针指向函数，通过这个指针调用函数。这个指针就成为函数指针。

声明函数指针的一般形式为：

类型 (*指针)(参数表)

函数指针的主要用途是在函数间传递函数。因为函数名本身不是变量，不能作为形参接收传递来的另一个函数。此时可以定义一个函数指针，使该指针指向要传递的函数。

例 4.17 编写一个函数，能完成各种有限级数求和并调用它求

$$g1(x) = \sum_{n=1}^5 x^n \quad \square$$

$$g2(x) = \sum_{n=1}^{10} (3\sin nx + 5\cos nx)$$

程序如下：

```

#include <iostream.h>
#include <math.h>
double sig(double (*fun)(int), int n, int m);    //函数的原型声明
double fg1(int k);                               //原型声明
double fg2(int k);                               //原型声明
double x;
void main()
{
    double g1,g2;
    cout<<"输入 x 的值:"; cin>>x;
    g1=sig(fg1, 1, 5);
    cout<<"g1="<<g1<<endl;
    g2=sig(fg2,1,10);
    cout<<"g2="<<g2<<endl;
}
double sig(double (*fun)(int), int n, int m)
{
    double g=0.0;
    for(int i=n; i<=m; i++) g+=(*fun)(i);        //用函数指针调用函数
    return g;
}
double fg1(int k)
{
    double s1=1;
    for(int i=1; i<=k; i++) s1*=x;
}

```

```

        return (s1);
    }
    double fg2(int k)
    {   double s2;
        s2=3*sin(k*x)+5*cos(k*x);
        return (s2);
    }

```

执行结果为:

```

输入 x 的值:2.0
g1=62
g2=1.92521

```

程序说明如下:

①程序中 sig() 函数的功能是求任一级数和。fg1()函数和 fg2()函数分别求 x^i 和 $3\sin(ix)+5\cos(ix)$ 。

②sig()函数的参数 (double (*fun)(int), int n, int m) 中 double (*fun)(int)是定义函数指针 fun, 接收的函数的返回类型是 double 类型, 且具有一个 int 类型的参数。

③main()函数两次调用 sig()函数, 分别求 g1 和 g2 两个函数的值。当 main()第一次调用 sig()函数时, 将 fg1 函数名给函数指针 fun, 在 sig()函数体内用 $g+=(fun)(i)$ 实现对 fg1 函数的调用, 每调用一次 fg1()函数, 求出 x^i 。sig()调用结束, 便求出 $x^1+x^2+\dots+x^n$ 。main()函数第 2 次调用 sig()函数时, 将 fg2 函数名给 fun 指针, 用 $g+=(fun)(i)$ 实现对 fg2 函数的调用, 调用后求得 $3\sin nx+5\cos nx$ 。

4.7 函数参数的缺省

在定义函数时, C++允许为函数的形参指定一个初值, 这个初值即为该参数的缺省值。以后在调用该函数时, 若省略了实际参数, C++自动按缺省值作为参数的值。缺省又称为默认。

例 4.18 函数参数的缺省。

```

#include <iostream.h>
int power(int a,int n=2);           //函数原型, 参数表中给出缺省值
void main()
{   int k(5);
    cout<<power(k)<<endl;
    cout<<power(k,4)<<endl;
}
int power(int a,int n)              //函数定义
{   int p=1;
    while(n>0) { p=p*a;  n--; }
    return p;
}

```

执行结果为:

```

25
625

```

程序说明如下:

①main()函数第1次调用 power(k)时, 实参 k 传递给形参 a, n 使用缺省值 2, 求出的是 $5^2=25$;第2次调用 power(k,4)时, 对应 a 的值仍是 k, n 的值为 4, 求出的是 $5^4=625$ 。

②也可以每个参数都有缺省值, 例如:

```
int power(int a=1, int n=2);
```

调用时若为 power(), 表示二个参数都用缺省值;若为 power(x), 表示 n 用缺省值, 而 a 用 x 的值。

③如果一个函数的参数表中有的参数有缺省值, 有的没有缺省值, 有缺省值的参数必须列在形式参数表右边。因为调用时要由左匹配参数。例如:

```
void func(int y, int x=2, int z=10);           //正确
```

```
void func(int x=2, int y, int z=10)           //错误
```

④若函数有原型声明又有定义时, 缺省值只需在原型声明中给出, 函数定义时的参数表中不能再给出, 见例 4.18。

4.8 函数重载

所谓函数重载, 是指两个以上的函数使用相同的函数名。但函数的形参必须有所不同, 可以是形参的个数不同, 或者形参的类型不同。调用重载函数时, 编译系统根据实参的个数或类型确定调用哪一个函数。

如果没有重载机制, 程序中的每个函数都必须有唯一的名字。例如, 要对整数、单浮点数和双浮点数分别做相加运算, 需要定义三个不同函数名的函数, 如

```
int addi(int x, int y);
```

```
float addf(float x, float y);
```

```
double addd(double x, double y);
```

在 C++ 中, 允许为这三个函数起一个共同的名字, 例如 add, 但它们的参数类型可以不同。编程时, 程序员只需记住 add 是两个数相加的函数, 具体调用时参数的匹配由编译系统完成。下面是实现两个数相加的函数重载程序。

例 4.19 参数类型不同的函数重载。

```
#include <iostream.h>
int add(int x, int y) { return x+y; }           //定义两整数相加
float add(float x, float y) { return x+y; }     //定义两单浮点数相加
double add(double x, double y) { return x+y; }  //定义两双浮点数相加
void main()
{ int i=100,j=200;
  float a=15.55, b=27.89;
  double x=6.3e10, y=8.05e12;
  cout<<"Integer:"<<add(i, j)<<"\n";
  cout<<"Float:"<<add(a, b)<<"\n";
  cout<<"Double:"<<add(x, y)<<"\n";
}
```

执行结果为:

```
Integer: 300
```

```
Float: 43.44
```

```
Double: 8.113e+012
```

例 4.20 参数个数不同的重载函数。

```

#include <iostream.h>
int max(int a, int b) { return a>b?a:b; }    //求 2 个数最大值函数
int max(int a, int b, int c)                //求 3 个数最大值函数
{
    int temp;
    temp=a>b?a:b;
    return temp>c?temp:c;
}
void main()
{
    int a,b,c;
    cout<<"输入 3 个数:"; cin>>a>>b>>c;
    cout<<"3 个数的最大值:"<<max(a,b,c)<<endl;
    cout<<"输入 2 个数:"; cin>>a>>b;
    cout<<"2 个数的最大值:"<<max(a,b)<<endl;
}

```

执行结果为:

```

输入 3 个数:23  5  42
三个数的最大值:42
输入 2 个数:12 9
二个数的最大值:12

```

使用重载函数要注意:重载函数的形参必须不同。如果函数名相同,形参类型和个数也相同,无论返回值的类型如何,编译时会出现函数重复定义的错误。另外,应注意避免因使用缺省参数值而引起函数重载时的二义性。

4.9 函数模板

模板是 C++ 支持参数化的工具。所谓参数化,就是将一段程序所处理的对象的类型参数化。这样,这段程序就可以处理允许范围内各种类型的对象。模板分为函数模板(function template)和类模板(class template)。这里介绍函数模板。

使用函数模板可以创建一个通用功能的函数,该函数支持多种不同的形参。

以下是两个求最大值的函数。函数定义如下:

```

int max(int x,int y) { return(x>y)?x:y; }
double max(double x, double y) { return(x>y)?x:y; }

```

这两个函数的功能相同,只是参数的类型和函数返回值的类型不同。为了减少程序代码的书写量,避免相同信息的重复,将上面程序段中的类型 `int` 和 `double` 参数化,即将 `int` 和 `double` 用一个参数 `T` 代替。这样,便得到下面通用的代码段:

```

T max(T x,T y) { return(x>y)? x: y; }

```

当要求某两个整数的最大值时,系统将通用代码段中的 `T` 换成 `int`;求两个实数的最大值时,将 `T` 换成 `double`,就可以得到前边的函数。这就是 C++ 中的函数模板。定义函数模板的一般形式为:

```

template <class  T>
类型  函数名(形参表)
      函数体

```

其中 `T` 代表在函数模板中使用的通用类型,既可以代表系统提供的基本数据类型,也可以代表用户自定义的类型。

函数模板不是一个完全的函数。它是一类函数的抽象，使用时将它的模板参数 `T` 实例化后，即可实现具体的函数。

例如，对上面求两个数最大值的函数定义一个函数模板的语句如下：

```
template <class T>
T max(T x,T y) { return(x>y)? x: y; }
```

在主函数中使用函数模板的语句如下：

```
void main()
{ int a(-15), b(23);
  double d1(50.678), d2(32.089);
  cout<<max(a,b)<<endl;
  cout<<max(d1,d2)<<endl;
}
```

在编译上述程序时，编译器从调用 `max(a,b)` 的实参中推导出函数模板中的 `T` 为 `int`，函数的返回值为 `int`；第 2 次调用 `max(d1,d2)` 时实参为双浮点型，从而推导出函数模板中的参数 `T` 为 `double`，函数的返回值为 `double`。

函数模板中的参数 `T` 可以有多个，例如：

```
template <class T1, class T2>
    函数定义
```

有两个参数，`T1` 和 `T2` 可以代表任意两个合法的数据类型。例如两个类型不同但个数相同的数组 `a` 和 `b` 相加，结果放 `a` 数组：

```
template <class T1,class T2>
void addarray(T1 a[ ], T2 b[ ],int size)
{ for(int i=0;i<size; i++) a[i]+=b[i];
  return;
}
```

在组织程序时，可以将函数模板放在程序的前部，一般常将函数模板的定义放在一个头文件中，以便作为一个通用函数使用。

函数模板是 C++ 替代 C 中带参数的宏的一种重要机制。函数模板比带参数的宏更加可靠、安全。因为对宏只进行正文替换处理，并不进行类型检查。而对函数模板，在调用时要进行对返回值及参数类型的严格检查。

学习模板是使用 C++ 中庞大的标准模板库 (STL) 的基础。

4.10 内联函数

在程序中调用定义的函数时，编译系统要把函数的参数进栈，函数运行完后要返回到调用语句的下一操作去执行。参数进出栈以及调用和返回等都会产生额外开销，尤其对多次调用的函数，降低了程序的运行效率。使用内联函数主要是为了提高程序的运行效率。

内联函数就是在定义函数的前面加上关键字 `inline`。C++ 编译程序碰到有关键字 `inline` 的函数时，就用函数体的代码替代函数的调用。例如：

```
#include <iostream.h>
inline float fss(float a) { return 3.1416*a*a; }
void main()
{ float x (50), y (40), s1, s2;
  s1=fss(x);
```

```

        s2=fss(y);
        cout<<s1<<" " <<s2<<endl;
    }

```

执行结果为:

```
7854    5026.56
```

在编译以上程序时，编译器在内联函数的调用处用内联函数体替换语句:

```

s1=fss(x);
s2=fss(y);

```

替换后成为:

```

s1=3.1416*x*x;
s2=3.1416*y*y;

```

对于一些规模较小多次调用的函数，使用内联函数可以减少程序的运行时间，但是增加了目标程序的长度。另外，使用内联函数还要注意以下几点:

①内联函数必须在调用之前声明或定义。因为内联函数的代码必须在替换之前已经生成，因此以下形式不会按内联函数处理:

```

float fss(float a);           //原型声明中没有声明为内联函数
void main()
{   float  x (50), y (40), s1, s2;
    s1=fss(x);
    s2=fss(y);
    cout<<s1<<" " <<s2<<endl;
}

inline float fss(float a ) { return 3.1416*a*a; }

```

②在内联函数中不能用循环语句和 switch 语句，如果内联函数中出现这些语句，编译系统仍作为普通函数来处理。另外，递归函数也不能作为内联函数。

4.11 系统函数

系统函数又称标准函数或库函数。C++编译系统将一些常用的操作以库函数的形式提供给用户。

4.11.1 系统函数概述

按功能不同，C++提供的系统函数分别放在不同的.h(头文件)文件中。例如，数学常用函数(如求指数函数、对数函数、绝对值函数、三角函数、平方根函数等)放在 `math.h` 文件中;字符处理函数(判断字母、数字、字母大小写等)放在 `ctype.h` 文件中;字符串处理函数(如求字符串长度、字符串连接、字符串拷贝、字符串比较等)放在 `string.h` 文件中;图形处理函数放在 `graph.h` 文件中。充分利用这些库函数，可以减少编程的工作量，提高程序的运行效率和可靠性。

使用系统函数时应注意以下问题。

①首先要了解 C++开发环境提供了哪些系统函数。不同的编译系统提供的系统函数不同，因此使用时需要查阅系统函数的参考手册或联机帮助，清楚每个函数的功能、名称、参数及函数返回值的使用方法，以便根据需要选用。常用库函数见附录。

②要了解系统函数在哪个头文件中。各头文件中给出了各类函数的原型说明。使用时，需在程序中用 `#include` 命令嵌入相应的头文件，否则调试程序时会连接不上，报连接错误。

4.11.2 字符串处理函数

1.求字符串长度函数 strlen()

该函数的原形是:

```
int strlen(char *str);
```

该函数的功能是返回 str 所指字符串的长度, 即字符串中遇到'\0'之前有效字符的个数, 但不包括字符串结束标志'\0'。

例 4.21 使用 strlen()函数计算字符串长度。

```
#include <iostream.h>
#include <string.h>
void main()
{   char s1[]="Hello C++";
    cout<<"s1:"<<strlen(s1)<<endl;
    char *s2="string";
    cout<<"s2:"<<strlen(s2)<<endl;
    char s3[]="Hello\0string";
    cout<<"s3:"<<strlen(s3)<<endl;
    cout<<"s4:"<<strlen("abcd xyz lmn")<<endl;
}
```

输出结果为:

```
s1: 9
s2: 6
s3: 5
s4: 12
```

程序共 4 次调用 strlen()函数, 每次给的参数形式不同。

2.字符串拷贝函数 strcpy()

该函数的原形是:

```
char *strcpy(char *str1, const char *str2);
```

其中 str1 可以是字符数组或字符指针, str2 可以是字符数组、字符指针或字符串常量。该函数的功能是将字符串 str2 复制到 str1 中。函数返回 str1 地址值。

调用该函数时注意, str1 所指向对象的容量应大于或等于 str2 的字符串长度, 否则将出现错误。

例 4.22 使用 strcpy()函数进行字符串拷贝。

```
#include <iostream.h>
#include <string.h>
void main()
{   char s1[]="C++ programming";
    char s2[20], *s3="Hello!";
    strcpy(s2,s1);
    cout<<"s2:"<<s2<<endl;
    strcpy(s2,s3);
    cout<<"s2:"<<s2<<endl;
    strcpy(s1+4, s3);
    cout<<"s1:"<<s1<<endl;
```

```

        strcpy(s2, "programming");
        cout<<"s2:"<<s2<<endl;
    }

```

输出结果为:

```

s2:C++ programming
s2:Hello!
s1:C++ Hello!
s2:programming

```

3.字符串连接函数 strcat()

该函数的原形是:

```
char *strcat(char *str1, const char *str2);
```

其中 str1 可以是字符数组或字符指针, str2 可以是字符数组、字符指针或字符串常量。该函数的功能是将 str2 字符串连接到 str1 字符串的后面, 去掉 str1 后边的'\0', 使两个字符串成为一个字符串。函数返回 str1 地址值。

使用该函数也要注意 str1 所指对象剩余空间应能容纳 str2 字符串, 否则会出错。

例 4.23 使用 strcat()函数进行两个字符串连接。

```

#include <iostream.h>
#include <string.h>
void main()
{   char s1[20]="Hello  ";
    char s2[]="everybody";
    strcat(s1, s2);
    cout<<"s1:"<<s1<<endl;
    char s3[20]="C++";
    strcat(s3, "programming");
    cout<<"s3:"<<s3<<endl;
    strcat(s3, "string");
    cout<<"s3:"<<s3<<endl;
}

```

输出结果为:

```

s1:Hello everybody
s3:C++programming
s3:C++programmingstring

```

4.字符串比较函数 strcmp()

该函数的原形是:

```
int strcmp(const char *str1, const char *str2);
```

其中 str1、str2 可以是字符数组、字符指针或字符串常量。该函数的功能是比较两个字符串, 函数的返回值是比较的结果。若 str1 和 str2 两个字符串相等, 函数返回值为 0;若 str1 大于 str2, 函数返回值为 1;若 str1 小于 str2, 函数返回值为-1。

两个字符串比较是按字符串中字符的 ASCII 值进行比较。比较的规则是:将 str1 与 str2 中的对应字符逐个进行比较, 直到出现两个字符不同或遇到字符串结束标志'\0'为止。

例 4.24 使用 strcmp()函数进行两个字符串比较。

```

#include <iostream.h>
#include <string.h>

```

```

void main()
{   char s1[20], s2[20 ]="abcdefg" ;
    cout<<"输入一个字符串(<20 个字符):";
    cin.getline(s1,20);                //获取以回车结束的一行字符
    int result=strcmp(s1, s2);
    if (result>0)   cout<<s1<<"大于"<<s2<<endl;
    else if (result<0)   cout<<s1<<"小于"<<s2<<endl;
    else   cout<<s1<<"等于"<<s2<<endl;
}

```

输出结果为:

```

输入一个字符串(<20 个字符):abcdxy
abcdxy 大于 abcdefg

```

再执行一遍程序:

```

输入一个字符串(<20 个字符):abcdefg
abcdefg 等于 abcdefg

```

程序中 `cin.getline(s1, 20)`表示调用输入流类成员函数 `getline`，最多输入 19 个字符并存入 `s1` 数组中。输入用回车结束，并把回车符自动转换为字符串结束标志符。

C++中提供的 `string` 类处理字符串会更加方便，有关内容见后续章节。

4.11.3 系统函数举例

例 4.25 从键盘输入一字符串，统计输出其中字母、数字、空格和其他字符的个数及字符总个数。

程序如下:

```

#include <iostream.h>
#include <ctype.h>                //字符函数头文件
#include <string.h> //字符串函数头文件
void main()
{   int alphas, digits, spaces, others;
    char s[80];
    alphas=digits=spaces=others=0;    //各种字符计数器
    cout<<"输入一串字符(可含空格):";
    cin.getline(s,80);                //输入以回车作为结束的一行字符
    int num=strlen(s);                //计算 s 中的字符个数
    for(int i=0; i<num; i++)
        if (isalpha(s[i])) alphas++;    //判断 s[i]中的字符是否为字母
        else if(isdigit(s[i])) digits++; //判断 s[i]中的字符是否为数字
        else if(isspace(s[i])) spaces++; //判断 s[i]中的字符是否为空格
        else others++;
    cout<<"字符总数为:"<<num<<endl;
    cout<<"所含字母个数为:"<<alphas<<endl;
    cout<<"所含数字个数为:"<<digits<<endl;
    cout<<"所含空格符个数为:"<<spaces<<endl;
    cout<<"其他字符个数为:"<<others<<endl;
}

```

执行结果为:

```
输入一串字符(可含空格):asd 759fdg3s6776hg,./jjjhu86  vuui,;*lkj
字符总数为:40
所含字母个数为:21
所含数字个数为:10
所含空格符个数为:3
其他字符个数为:6
```

本例中函数 `isalpha()`、`isdigit()`和 `isspace()`均属字符函数，它们的原型说明出现在 `ctype.h` 文件中。其中 `isalpha()`判断 `s[i]`中的字符是否是字母，是字母则函数返回 1，否则返回 0。`isdigit()`和 `isspace()`分别判断 `s[i]`中的字符是否是数字或空格。

例 4.27 用梯形法求定积分 $\int_a^b f(x)dx$ 的值。

定积分 $\int_a^b f(x)dx$ 的几何意义是求曲线 $f(x)$ 和 $x=a$ 、 $x=b$ 和 $y=0$ 所围成的曲边梯形面积。现将该面积分成若干个小的曲边梯形，并用若干个小梯形面积之和近似代表定积分的值。已知任意函数定积分的梯形公式为:

$$\int_a^b f(x)dx \approx \left[\frac{1}{2} (f(a) + f(b)) + \sum_{i=1}^{n-1} f(a+ih) \right] h$$

其中， n 为等分区间 $[a,b]$ 时的小区间个数， n 值较大时更接近原定积分的值; $h=\frac{b-a}{n}$ 为小梯形的高。设 n 的值由键盘输入。求本题定积分的值的程序如下:

```
#include <iostream.h>
#include <math.h>
double fx(double x);
void main()
{ double a, b, y, h;
  int n, i;
  a=0.0; b=2.0;
  cout<<"输入 n 的值:"; cin>>n;
  h=(b-a)/n;
  y=(fx(a)+fx(b))/2.0;
  for(i=1; i<n; i++) y=y+fx(a+i*h);
  y=y*h;
  cout<<"y="<<y<<endl;
}
double fx(double x) { return (sin(x)+exp(-x)); }
```

执行结果为:

```
输入 n 的值:100
y=2.28079
```

4.12 作用域、生存期与可见性

程序中的每个标识符(如变量名、函数名、标号等)都有各自的作用域、生存期和可见性。

4.12.1 作用域

一个标识符在源程序正文中有意义的那个区域称为作用域。C++中有 5 种作用域，即文件作用域、块作用域、函数作用域、函数原型作用域和类作用域。此处仅介绍前 4 种作用域。

1. 文件作用域

任何在函数之外定义或说明的标识符都有文件作用域。从定义这些标识符的位置开始直到文件末尾，任何函数都可以访问这些标识符。

例如，下面一段程序中的变量 `i` 和函数 `fun1()` 及函数 `fun2()` 原型的作用域为文件作用域：

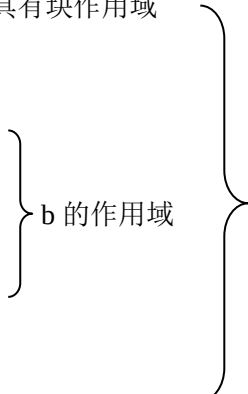
```
int i;                                //全局变量 i
inf fun1(){...} //函数体略
void fun2(int &,int *);
void main() {...}
```

上述变量 `i` 常称为全局变量。

2. 块作用域

为了说明块作用域的概念先看一个例子：

```
void fun(int x)                //形参 x 具有块作用域
{ int a;
  if(x>0) a=x;
  else { int b;
        cout<<"输入一个整数:";
        cin>>b;
        a=x+b*b;
      }
  cout<<"a="<<a<<endl;
}
```



以上程序中，在函数体开头定义了变量 `a`，在 `if` 语句的 `else` 后面的复合语句中又定义了变量 `b`。变量 `a` 和变量 `b` 都具有块作用域，但为不同的块作用域。块是一对花括号括起来的一段程序。函数体是一个块，复合语句也是一个块，二者是包含关系。复合语句中还可嵌套复合语句。

在块内定义或说明的标识符的作用域从定义或说明处开始，一直到块结束的花括号为止。注意，函数的形式参数 `x` 也具有块作用域，即该函数的函数体。

具有块作用域的标识符也称为局部标识符。如上例中的变量 `a`、`b` 和形参 `x` 都是局部变量。

3. 函数作用域

只有语句标号有函数作用域，即仅在语句标号所出现的函数内有效。

4. 函数原型作用域

只有在函数原型参数表中使用的标识符才具有函数原型作用域。在函数原型说明中，不要求写出形参的名称，只要求给出类型。如果函数原型说明中给出形参名称，则编译程序会忽略这些名称。所以，这些形参的标识符可以在程序中的其他地方使用，而不会产生二义性。也就是说，函数原型作用域仅是在函数原型说明中的左、右圆括号之间。例如有以下函数原

型:

```
void function(double xx, double yy);
```

其中的 `xx` 和 `yy` 只在所说明的括号内有效，在程序的其他地方使用 `xx` 和 `yy` 时必须进行定义。

4.12.2 生存期

生存期是指程序运行时一个变量或数组从给它分配内存空间到释放所占空间经历的期间。它与作用域相关，但又有区别。作用域是编译期间才有的概念。在 C++ 中，生存期可以用标识符的存储类别修饰符指定。

1. 静态生存期

任何一个具有文件作用域的全局变量、函数以及全体常数的生存期都有静态生存期。静态生存期与整个程序的运行期一致，即程序开始执行时为它们分配存储空间，程序执行结束前释放它们。

对于静态生存期的标识符，缺省时的存储类别为 `extern`。如果要使一个非文件作用域的标识符的生存期为静态的，则需指定存储类别为 `static`。例如：

```
static int radius;
```

此处，变量 `radius` 将具有静态生存期，因此称它们为静态变量。若静态变量定义在任何函数或类之外，则称为外部静态变量；若定义在某个函数之内，则称为内部静态变量或局部静态变量。

局部静态变量的生存期是从第一次执行其定义的语句开始，直到整个程序执行结束。所有具有静态生存期的变量和数组都仅被初始化一次，且缺省的初始值为 0。

2. 局部生存期

除了具有静态生存期的变量、数组外，其余的变量、数组等都具有局部生存期。具有局部生存期的变量、数组常被称为局部变量、局部数组。它们的存储类别缺省为 `auto` (实际上也很少明确写出)。

具有局部生存期的变量和数组只在程序运行中遇到它们的定义或说明时才在栈中为它们分配存储空间，在它们的作用域结束处把它们占用的存储空间释放。

直到目前为止，在程序中定义或说明的变量、数组和形参都是具有局部生存期的局部量。

C++ 中还允许使用存储类别修饰符 `register`，用 `register` 修饰的变量具有局部生存期。这种变量又称为寄存器变量，表示它们将可能存储到 CPU 的寄存器中，以便加快程序的执行速度。例如：

```
register int index;
```

寄存器变量只能是具有块作用域的 `int` 型或 `char` 型变量或形式参数。寄存器变量不允许进行取变量地址的操作。当编译优化后，可能会将某些经常使用的 `auto` 型变量存入寄存器中。同时，如果没有足够多的寄存器可用，编译程序会自动将 `register` 型变量转为 `auto` 型变量。

3. 动态生存期

使用运算符 `new` 在堆中分配的变量或数组具有动态生存期。具有动态生存期的变量或数组必须用运算符 `delete` 释放它们占用的堆中的存储空间。

4.12.3 可见性

在一般情况下，标识符的可见性与其作用域相同或在其作用域之内。它是从另一个角度表现标识符的有效性。标识符在某个位置可见，表示该标识符可以被引用。

1. 具有文件作用域的标识符的可见性

具有文件作用域的函数、变量或数组的可见性是从其定义域或说明之处开始，直到它所在的源程序文件结束为止。

如果要在其他源程序文件中引用具有文件作用域的函数，若先编译这些函数所在的源程序文件，则不必加函数原型说明，否则必须加函数原型说明。

如果在其他源程序文件中引用这些全局变量、数组，或者要引用在同一源程序的后面定义的全局变量或数组，则应用 **extern** 存储类别修饰符给予说明，以扩大可见性范围。

例 4.29 引用在同一源程序文件后面定义的全局变量。

```
#include <iostream.h>
#define N 10
double a;                //定义全局变量 a
void main()
{   extern double x[N];    //引用在后面定义的全局数组
    for(int i=0; i<N; i++)  a=a+x[i];
    cout<<"a="<<a<<endl;
}
double x[N]={0.1, 1.1, 2.1, 3.1, 4.1, 5.1, 6.1, 7.1, 8.1, 9.1};
```

执行结果为:

```
a=46
```

在该程序中，变量 **a** 和数组 **x** 定义在函数体外，是具有文件作用域的全局变量。对于 **a** 可以在 **main()** 函数中直接引用，但要在 **main()** 函数中引用在 **main()** 函数之后定义的数组 **x**，就必须在 **main()** 函数中引用数组 **x** 之前说明它。

例 4.30 引用另外源程序文件中的全局变量和函数。

```
//file1.cpp 文件
#include <iostream.h>
int a=5;
extern int b, fun1(),c;
extern void fun2();
void main()
{   cout<<"main():a="<<a--<<"b="<<b++<<endl;
    cout<<fun1()<<endl;        //调用 fun1()函数
    c=2;
    fun2();                    //调用 fun2()函数
}
//file2.cpp 文件
#include <iostream.h>
extern int a;
int b=10;
int fun1()
{   cout<<"fun1():a="<<a++<<"b="<<--b<<endl;
    return a-b;
}
//file3.cpp 文件
#include <iostream.h>
int c=6;
```

```

extern int fun1();
void fun2()
{   extern int a,b;
    cout<<"fun2():a="<<a<<"b="<<b<<"c="<<c<<endl;
    a=4; b=0;
    cout<<fun1()<<endl;
}

```

输出结果为:

```

main():a=5 b=10
fun1():a=4 b=10
-5
fun2():a=5 b=10 c=2
fun1 ():a=4 b=-1
6

```

在例 4.30 中, 整个 C++ 源程序由三个源文件组成。在 file1.cpp 中定义全局变量 a。file2.cpp 中要引用 file1.cpp 源文件中的全局变量 a, 需要用 extern 进行说明。file1.cpp 中要引用另外文件中定义的变量 b 和函数 fun1()、fun2(), 需要用 extern 进行说明。类似地, 在 file2.cpp 和 file3.cpp 源文件中也均有自身定义的全局变量和引用其他源文件全局变量和函数的 extern 说明。只有这样, 才能扩大在其他源文件中定义的全局变量和函数的可见性范围。

2. 具有块作用域的变量的可见性

具有块作用域的变量、数组的可见性为从其在块内定义或说明之处开始, 直到所在的块结束为止。

例 4.31 具有块作用域变量的可见性。

```

#include <iostream.h>
void main()
{   int a=10;
    cout<<"a="<<a<<endl;
    {   int b=20;
        a=a+b;
        cout<<"a="<<a<<endl;
        cout<<"b="<<b<<endl;
        a=15;
    }
    cout<<"a="<<a<<endl;
}

```

输出结果为:

```

a=10
a=30
b=20
a=15

```

在例 4.31 程序中, 变量 a 的可见性在整个函数体内, 而 b 的可见性仅在一复合语句中。

3. 不同作用域的同名标识符的可见性

当具有不同作用域的标识符同名时, 将按“局部优先”的原则确定它们的可见性。

例 4.32 不同作用域的同名标识符的可见性。

```

#include <iostream.h>
int a=10;                                //定义全局变量 a
void main()
{   cout<<"全局变量 a="<<a<<endl;
    float a=7.8;                        //定义函数体内局部变量 a
    cout<<"main 中 a="<<a<<endl;
    { int a=5;                          /定义复合语句中局部变量 a
      cout<<"复合语句中 a="<<a<<endl;
      a=15;
    }
    cout<<"a="<<a<<endl;
}

```

输出结果为:

```

全局变量 a=10
main 中 a=7.8
复合语句中 a=5
a=7.8

```

从输出结果可以看出:在 main()函数中没有定义变量 a 之前,输出的 a 是全局变量 a 的值;在 main()块作用域内定义 float 类型变量 a 后,输出的 a 是刚刚定义的局部变量 a;而在复合语句中,全局变量 a 和函数块作用域的局部变量 a 均不可见,只能见到在复合语句内定义的局部变量 a。这种现象称为名称隐蔽或名称优先性。

4.静态类别标识符的可见性

若一个函数被指定为静态类别时,该函数只能被它所在的源程序文件中的其他函数调用,而不能被另外的源程序文件中的函数调用,即可见性仅存在于出现其定义的源程序文件内。若引用在先而定义在后,则需加函数原型说明。例如,将函数 fmax()定义为静态函数:

```
static int fmax(int x, int y) { return x>y?x:y; }
```

函数 fmax()只允许同一程序文件中的函数调用。

外部静态变量的可见性仅限于它所在的源程序文件内,且从定义或说明之处开始,直到源程序文件结束为止的范围内。

内部静态变量的可见性仅限于它所在的程序块内。当它所在的函数被调用时,执行到它的定义之处开始可见,直到它所在的块结束处为止。而且,在定义它的作用域外,虽然不可见,但它仍然存在,没有被释放。它的值保持到下次被调用并被改变之前。利用这一特点,既避免了全局变量可能被无意中修改的缺点,又保持了其值可在局部反复被引用的优势。因此,在编制大型程序时经常使用内部静态变量。

内部静态变量有默认值, int 型为 0, 浮点型为 0.0, char 为空。

例 4.30 内部静态变量的使用。

```

#include <iostream.h>
int sum(int p[ ],int &a);
void main( )
{   int data[ ][3]={1,2,3},{4,5,6},{7,8,9}}, s;
    for(int i=0; i<3; i++)
        { cout<<sum(data[i],s)<<" "; cout<<s<<endl; }
}
int sum(int p[ ], int &a)

```

```

{   int i, k=0;
    static int s=0;                //定义 s 为局部静态变量
    for(i=0; i<3; i++) { s+=p[i];  k+=p[i]; }
    a=k;
    return s;
}

```

输出结果为:

```

6  6
21 15
45 24

```

例 4.33 的函数 `sum()` 中定义了一个局部静态变量 `s`，初值为 0(静态生存期的变量仅初始化一次);还定义了一个局部变量 `k`，初值也为 0(在每次调用局部变量所在的函数 `sum` 时，均重新对局部变量 `k` 进行初始化)。

尽管 `s` 与 `k` 的运算完全相同，但由于 `s` 是静态变量，它保存了函数 `sum()` 在上一次调用后计算的结果:第 1 次调用后其值为 6;第 2 次调用 `sum()` 时在原值 6 的基础上进行累加计算，调用后其值变为 21;第 3 次调用 `sum()` 时，使用上次调用后的结果 21 进行计算，调用后其值变为 45。`s` 得到的是数组 `data` 中所有元素的和。而内部变量 `k` 每次调用时均赋初值 0，调用后释放其所占存储空间，故每次计算的只是 `data` 数组中某一行元素值之和。

表 4.1 总结了全局变量、局部变量和静态变量的位置及生存期。

表 4.1 变量作用域、生存期与可见性

作用域	声明位置	生存期	可见性
局部	函数体内、程序块内	函数或块被执行时存在，函数调用结束时被释放。	声明的函数体内、块内
内部静态	函数体内	被初始化一次，一直到程序结束	函数体内
外部静态	函数体外	从声明处开始，直到所在源程序文件结束	它所在的源程序文件内
全局	函数体外	从声明处开始，一直到程序结束	整个程序，不同文件或定义在后时，用 <code>extern</code> 说明

4.12.4 作用域分辨符(::)

为了使全局变量处处可见，可以使用作用域分辨符“::”。

例 4.34 作用域分辨符的使用。

```

#include <iostream.h>
int a;
void main()
{   int a=10;
    {   int a=3;
        ::a=5;
        a=::a+a;
    }
}

```

```

        cout<<"::a="<<::a<<" a="<<a<<endl;
    }
    ::a++;
    cout<<"::a="<<::a<<" a="<<a<<endl;
}

```

输出结果为:

```

::a=5  a=8
::a=6  a=10

```

从上例可以看出,作用域分辨符“::”提供了在局部内引用与局部变量同名的全局变量的方法。若不用“::”符号,则同名的全局变量被隐蔽起来。但要注意,该运算符只能用于访问全局变量(将来还要介绍用于访问某类的成员),不能用于访问任何具有块作用域的局部变量。

4.13 编译预处理

C++编译系统在对源程序进行编译之前先对一些特殊的命令进行“预处理”,然后再将预处理的结果和源程序一起进行通常的编译处理。预处理功能由 C++语言提供的预处理命令实现。常用的编译预处理命令有三种,即包含文件、宏定义和条件编译。预处理命令以一个“#”开始。“#”一般在程序第 1 列位置。通常每条预处理命令单独占一行,预处理命令不是程序中的语句,所以预处理命令行的最后没有“;”。

预处理的过程是个直接的符号代换过程。预处理并未改写源程序文件本身,但它可以改善程序的组织和管理,帮助程序设计人员编写易读、易改、便于调试和便于移植的程序。

4.13.1 包含文件(#include)

包含文件有两种格式:

```
#include <文件名>(第一种格式)
```

```
#include "文件名"(第二种格式)
```

第一种格式命令中的文件名一般是系统提供的系统头文件(头文件扩展名为.h),这种文件一般存放于 C++系统目录中的 include 子目录下。编译系统处理这条命令时,就在 include 子目录下搜寻给出的文件并把它嵌入到当前文件中。这种方式是标准方式。

编译系统处理第二种格式时,首先在当前文件所在的目录中寻找,如果找不到,再按标准方式寻找。这种方式一般适用于包含用户自己建立的头文件。

包含文件可以嵌套,既在包含文件中还可以有包含命令。

4.13.2 宏定义(#define)

宏定义命令是将一个标识符定义为一个字符串。被定义的标识符称为宏名,定义的字符串叫做替换文本。宏定义有两种形式,即不带参数的宏定义和带参数的宏定义。

1. 不带参数的宏定义

不带参数的宏定义的形式为:

```
#define 宏名 替换文本
```

其中,宏名是用户定义的标识符,替换文本可以是任何形式的常量或字符串。例如:

```

#define M 10
#define PI 3.14159
#define FS "****"

```

在编译预处理时，编译系统把程序中出现的宏名一律用替换文本替换，然后再对替换后的源程序文件进行编译。替换又称宏展开。

例 4.35 宏定义命令使用，分析程序运行结果。

```
#include <iostream.h>
#define N 5
#define pr cout<<endl
void main()
{   int i,j;
    int a[N][N]={ {1,2,3,4,5},{2,3,4,5},{3,4,5},{4,5},{5}};
    for(i=0; i<N; i++)
    { for(j=0; j<N-i; j++)
      cout<<a[i][j]<<" ";
      pr;
    }
}
```

执行结果为:

```
1  2  3  4  5
2  3  4  5
3  4  5
4  5
5
```

以上程序经编译预处理后，主函数中的 N 替换为 5，pr 替换为 cout<<endl。编译程序对替换后的源程序进行分析处理并生成相应的目标程序。

2.带参数的宏定义

带参数的宏定义的形式为:

`#define 宏名(参数表) 替换文本`

其中参数表是用逗号隔开的一些标识符。参数表中的各参数应出现在替换文本中。程序中引用宏定义时，除给出宏名外，宏名后给出用一对括号括起来的对应于定义中参数表的实际量。例如，有以下宏定义:

```
#define pow(x,y) (x)*(x)+(y)*(y)
```

在程序中使用已定义的宏，例如:

```
int a=10, b=15;
x=pow(a+b, a-b);
```

预处理后，宏替换为:

```
x=(a+b)*(a+b)+(a-b)*(a-b);
```

设计程序时，可以将一些多次使用的表达式定义为带参数的宏，从而简化程序。但在使用宏时一定要特别注意，因为编译程序在展开宏时，并不对定义的宏进行语法和语义检查，而只是原样替换。例如，若将上面定义的宏名 pow 定义为

```
#define pow(x, y) x*x+y*y
```

仍在程序中用以下形式调用:

```
x=pow(a+b,a-b);
```

预处理后的展开式为:

```
x=a+b*a+b+a-b*a-b;
```

因此定义时要根据情况适当地加括号，以保证不会出现逻辑错误。

例 4.36 分析以下带参数的宏定义程序运行结果。

```
#include <iostream.h>
#define SR(x,y) x+y
void main()
{   int a(5), b(4);
    a+=2*SR(a,b);
    cout<<"a="<<a<<endl;
}
```

运行结果为:

a=19

由于用 `const`(见第 1 章)定义的常量有类型含义,在调试程序中可见,而且可以作为参数使用,使用更加方便,而`#define` 只是简单的代换。所以在 C++中宏定义使用较少。

4.13.3 条件编译

条件编译是指根据给定的条件决定对程序中的某些程序段是否进行编译。当条件满足时,相应的程序段被编译并产生目标代码,否则不被编译。条件编译命令有:

`#if` `#else` `#elif` `#endif` `#ifdef` `#ifndef` `#undef`

一般使用形式如下:

```
#ifdef 标识符
    程序段 1
#else
    程序段 2
#endif
```

可以在程序的中间插入条件编译。该条件编译命令的作用是:如果标识符已被定义,对程序段 1 进行编译;否则对程序段 2 进行编译。其中程序段 1 和程序段 2 可以是由任意一组 C++语句组成的程序段。其中的`#else` 和程序段 2 也可以省略。

例 4.37 使用条件编译命令,根据条件将小写英文字母转换为大写英文字母。

```
#define capital 0
#include <iostream.h>
void print(char str[]);
void main()
{   char s[]="aBcdEf mNg";
    print(s);
}

void print(char str[])
{   char c;int i=0;
    while((c=str[i])!='\0'){
        #if capital
            if(c>='a' && c<='z') c-='a'-'A';
        #endif
        cout<<c;
        i++; }
}
```

执行结果为:

aBcdEf mNg

当#define capital 1 时，执行结果为:

ABCDEF MNG

该程序根据标识符 capital 是 1 或 0 决定是否将 str 数组所存字符串中的小写字母转换成大写字母。当需要将字符串中小写字母转换成大写字母时，将 capital 定义为 1，编译系统根据 capital 的值将 if 语句进行编译，完成小写转换成大写;否则编译系统不处理 if 语句，程序将字符串内容原样输出。

4.14 例题

例 4.38 将输入的字符串倒序输出。

```
#include <iostream.h>
#include <string.h>
#define N 80
void rever(char *s);
void main()
{   char string [N];
    cout<<"输入一个字符串:";
    cin.getline(string, 80);
    rever(string);                //调用 rever 函数
    cout<<"倒序后的字符串:";
    cout<<string<<endl;
}
void rever(char *s)
{   char *t;
    int c;
    for(t=s+(strlen(s)-1); s<t; s++, t--) { c=*s; *s=*t; *t=c; }
}
```

运行结果为:

输入一个字符串:abcdefghijkl

倒序后的字符串:lkjhgfedcba

例 4.39 判断输入的一个数是否为素数。

```
#include <iostream.h>
int GetNum();
void IsPrime(int);
void main()
{   int number;
    number=GetNum();
    IsPrime(number);
}
int GetNum()
{   int num, ok;
```

```

do{ ok=0;
    cout<<"请输入一个正整数:"; cin>>num;
    if(num<=0){ cout<<"输入数据有误，请重新输入!\n"; ok=1; }
}while(ok );
return num;
}
void IsPrime(int number)
{ int result, ctr=2;
  while(ctr<number)
  { result=number%ctr;
    f(result==0){ cout<<"可以被"<<ctr<<"整除，不是素数!\n"; return; }
    ctr++;
  }
  cout<<"\n 整数"<<number<<"是素数!\n";
  return;
}

```

例 4.40 实现字符串按字典顺序排序。

```

#include <iostream.h>
#include <string.h>
#define N 6
void sort(char **pp);
void main()
{ char *name[N]={“binary”,“address”,“parallel”,“interrupt”,“digital”,“check”};
  sort(name);
  cout<<"排序结果:\n";
  for(int i=0; i<N; i++) cout<<name[i]<<endl;
}
void sort(char **pp)
{ char *temp;
  int i,j,k;
  for(i=1; i<=N-1; i++)
  { k=N-i;
    for(j=0; j<=k-1; j++)
      if(strcmp(pp[j],pp[j+1])>0)
        { temp=*(pp+j); *(pp+j)=*(pp+j+1); *(pp+j+1)=temp; }
  }
}

```

执行结果为:

```

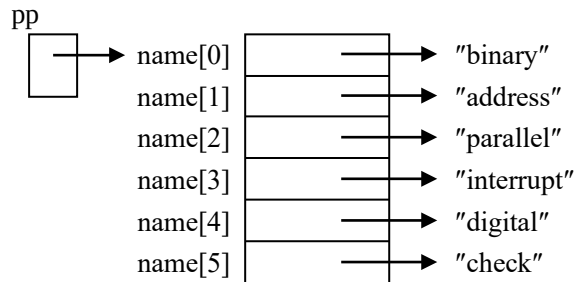
address
binary
check
digital
interrupt
parallel

```

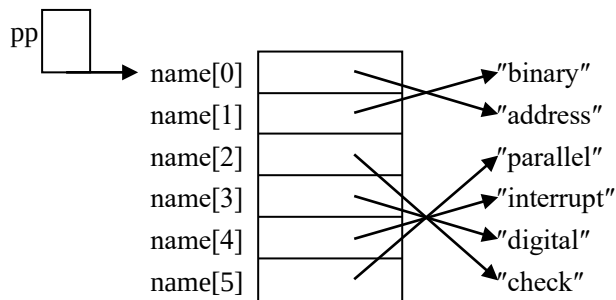
程序说明如下:

①该程序采用冒泡排序法对字符串进行排序。

②main()函数调用 sort()函数时, 将指针数组 name 名作为实际参数。name 中各元素存放的是各字符串的首地址。sort()函数的形参中用二级指针 pp 接收, 所以关系为:



③sort()函数中 temp 为一指针。当条件 if (strcmp(*pp[j], *pp[j+1])>0)满足时, 交换的是 pp 指针所指内容, 即 name 数组中的内容, name 数组中存放的是字符串的地址, 所以交换的是指针。执行后的关系为:



练习 4

1.回答下列问题:

- (1)什么是函数的值和函数的类型, 是否所有函数都有返回值?
- (2)return 语句的作用是什么, 一个函数中是否可以有多个 return 语句?
- (3)函数原型和函数调用有何区别?
- (4)函数参数的传值调用和引用调用有什么区别?
- (5)函数调用的实参是一维数组名时, 形参可以是什么形式?
- (6)二维数组做参数时, 形参有哪几种形式?
- (7)什么是函数重载, 实现重载的原则是什么?
- (8)什么是函数参数的缺省值(默认值), 使用参数缺省值有什么注意事项?
- (9)什么是内联函数, 使用内联函数的作用是什么?
- (10)函数的嵌套调用和递归调用有什么不同?
- (11)函数模板的作用是什么, 说明函数模板的使用形式。
- (12)什么是标识符的作用域、生存期和可见性?
- (13)什么是局部变量和全局变量?
- (14)内部静态(局部静态)类变量和外部静态类变量有何区别?
- (15)局部静态变量和局部自动变量有何区别?
- (16)说出数字、字符、字符串和输入输出函数所在的包含文件名。

2.写出以下程序的运行结果:

- (1)

```
#include <iostream.h>

int function(int x);

void main()
{ cout<<"function(7)<<endl; }

int function(int x)
{   int s;
    if( x==1||x==2) return 2;
    s=x*function(x-2);
    return s;
}
```
- (2)

```
#include <iostream.h>

double a;

void main()
{ int a=10;
  { int a=20;
    double b;
    ::a=20.5;
    b::a+a;
    cout<<"a"<<a<<"   "<<"b"<<b<<endl;
    cout<<"a"<<::a<<endl;
  }
  cout<<"a"<<::a<<endl;
}
```
- (3)

```
#include <iostream.h>
```

```

void sub(int x,int y,int &z){ z=y*x;}
void main()
{   int a,b,c;
    sub(10,5,a);   sub(8,a,b); sub(a,b,c);
    out<<a<<"   "<<b<<"   "<<c<<endl;
}

```

(4) #include <iostream.h>

```

void print(int n);
void main()
{   for(int i=1; i<6; i++) print(i); }
void print(int n)
{   for(int i=1; i<=n; i++) cout<<"**";
    cout<<endl;
}

```

(5)#include <iostream.h>

```

template <class T>
void fx(T a[], int n)
{ T temp;
  for(int i=0; i<n/2;i++){ temp=a[i]; a[i]=a[n-i-1]; a[n-i-1]=temp; }
  return;
}
void main()
{   int a[6]={1,2,3,4,5,6};
    double x[6]={2.2,3.3,4.4,5.5,6.6,7.7 };
    fx(a,6); fx(x,6);
    for(int i=0; i<6; i++) cout<<a[i]<<"   "<<x[i]<<endl;
}

```

(6) #include <iostream.h>

```

#include <string.h>
void function(char s[];
void main()
{   char str[10]="abcd";
    cout<<str<<endl;
    int n1=1, n2=strlen(str);
    while(n1++<=n2)   function(str) ;
    cout<<str<<endl;
}
void function(char s[])
{   static int n3=0;
    s[n3++]+=2;
}

```

3.编制 4 个函数，分别求两数和的平方、两数平方的和、两数差的平方、两数平方的差。
在主函数中分别调用以上 4 个函数，并输出任意两数的 4 种结果。

4.编写两个函数分别求 $n!$ 和 $\sum_{i=1}^n i$ 。main()调用这两个函数，分别输出 s1 和 s2。其中：

$$s1 = \frac{m!}{n!(m-n)!}$$

$$s2 = \sum_{i=1}^m i - \sum_{j=1}^n j \quad (\text{其中 } m、n \text{ 为正整数且 } m>n)$$

5.编写函数，统计一字符串中数字字符占有所有字符的百分比。

6.分别用矩形法和梯形法求定积分：

$$y = \int \pi 0(\sin x + \cos x) dx$$

当 n 取值相同时，比较两种方法的精确度。

7.编写函数，分别求两个整数的最大公约数和最小公倍数，并在 main()函数中任意输入两个整数，调用以上定义的函数并输出结果。

8.定义一个求绝对值函数的模板，调用它分别求整数、单浮点数和双浮点数的绝对值。

9.将第 8 题用 3 个重载函数实现。

10.编写一个函数，对任意给的三个边 a 、 b 、 c 判断是否能构成三角形。如能构成三角形，则用公式求三角形的面积。面积公式为：

$$area = \sqrt{p(p-a)(p-b)(p-c)}$$

其中：

$$p = \frac{a+b+c}{2}$$

构成三角形的判断条件是： $a+b>c$ 而且 $|a-b|<c$ 。

11.编写一个函数 delchar(char *s1, char s2)，从字符串 s1 中删去与字符 s2 相同的字符。用 main()测试该函数。

12.编写一个能对任意类型(整型、字符型、浮点型)数据进行排序(由小到大)的通用函数，并用 main()测试该函数。

13.编写一个函数 function(int n, int k)。该函数有 2 个参数 n 和 k 。函数的返回值为数 n 从右边开始数的第 k 个数字的值。例如 $n=1234$ ， k 为 2 时，函数返回 3。若 $k>n$ 的位数或 $k<1$ 时，函数返回 0。

14.编写求二维数组中每行元素的平均值的函数。调用该函数，求 array[5][4]每行元素的平均值。

15.编写函数 int find(char a[])，从一个字符串中找出某个字符第一次出现的位置，例如字符串“Computer Language”中字母“u”第 1 次出现的位置为 5。若没有该字符，返回 0；否则函数返回出现的位置。

16.编写函数，该函数能完成任意的十进制数转换为 $R(2 \leq R \leq 16)$ 进制数。调用该函数，将输入的一个十进制正整数分别转换为 R 进制数数字串。

17.设有 30 个学生参加考试，学号由 1 到 30。分别编写以下函数：

(1)将考试成绩大于 85 分的学生的学号和成绩输出；

(2)将 60 分以下的学生的学号和成绩输出；

(3)统计大于或等于平均分的人数。

(4)将 30 个学生成绩按由高分到低分排列，输出排列好的成绩及每个成绩对应的学号。